HACETTEPE UNIVERSITY

ELECTRICAL AND ELECTRONICS ENGINEERING

ELE432 - Advanced Digital Design

HOMEWORK 2-Multicycle Processor Controller

Name & Surname: BERKE MALÇOK

Student ID: 2210357081

,

Q1)

I spent approximately 12 hours completing this lab. The most time-consuming part was analyzing the synchronization between the testbench's clock edges and the FSM state transitions to ensure the outputs matched the expected test vectors perfectly.

Q2)

The full hierarchical design consists of the **controller (top module), maindec (FSM), aludec (ALU Control), and instrdec (Immediate Decoder).**

code

CONTROLLER

```
module controller(
   input  logic      clk,
   input  logic      reset,
   input  logic [6:0] op,
   input  logic [2:0] funct3,
   input  logic      funct7b5,
   input  logic      zero,
   output logic [1:0] immsrc,
   output logic [1:0] alusrca,
   output logic [1:0] alusrcb,
   output logic [1:0] resultsrc,
   output logic      adrsrc,
   output logic [2:0] alucontrol,
   output logic      irwrite,
   output logic      pcwrite,
   output logic      regwrite,
   output logic      memwrite
);

   logic [1:0] aluop;
   logic      branch;
   logic      pcupdate;

   maindec md(
      .clk(clk),
      .reset(reset),
      .op(op),
      .alusrca(alusrca),
      .alusrcb(alusrcb),
      .resultsrc(resultsrc),
      .adrsrc(adrsrc),
```

```
        .irwrite(irwrite),
        .regwrite(regwrite),
        .memwrite(memwrite),
        .branch(branch),
        .pcupdate(pcupdate),
        .aluop(aluop)
    );

    aludec ad(
        .opb5(op[5]),
        .funct3(funct3),
        .funct7b5(funct7b5),
        .aluop(aluop),
        .alucontrol(alucontrol)
    );

    instrdec id(
        .op(op),
        .immsrc(immsrc)
    );

    assign pcwrite = pcupdate | (branch & zero);

endmodule
```

MAINDEC

```
module maindec(
    input  logic       clk,
    input  logic       reset,
    input  logic [6:0] op,
    output logic [1:0] alusrca,
    output logic [1:0] alusrcb,
    output logic [1:0] resultsrc,
    output logic       adrsrc,
    output logic       irwrite,
    output logic       regwrite,
    output logic       memwrite,
    output logic       branch,
    output logic       pcupdate,
    output logic [1:0] aluop
);

    typedef enum logic [3:0] {
        S0_FETCH    = 4'd0,
        S1_DECODE   = 4'd1,
        S2_MEMADR   = 4'd2,
        S3_MEMREAD  = 4'd3,
        S4_MEMWB    = 4'd4,
```

```
    S5_MEMWRITE = 4'd5,
    S6_EXECUTER = 4'd6,
    S7_ALUWB    = 4'd7,
    S8_EXECUTEI = 4'd8,
    S9_JAL      = 4'd9,
    S10_BEQ     = 4'd10
  } statetype;

  statetype state, nextstate;

  always_ff @(posedge clk or posedge reset) begin
    if (reset)
      state <= S0_FETCH;
    else
      state <= nextstate;
  end

  always_comb begin
    case (state)
      S0_FETCH: nextstate = S1_DECODE;

      S1_DECODE: begin
        case (op)
          7'b0000011: nextstate = S2_MEMADR;   // lw
          7'b0100011: nextstate = S2_MEMADR;   // sw
          7'b0110011: nextstate = S6_EXECUTER; // R-type
          7'b0010011: nextstate = S8_EXECUTEI; // I-type ALU
          7'b1101111: nextstate = S9_JAL;      // jal
          7'b1100011: nextstate = S10_BEQ;     // beq
          default:    nextstate = S0_FETCH;
        endcase
      end

      S2_MEMADR: begin
        case (op)
          7'b0000011: nextstate = S3_MEMREAD;   // lw
          7'b0100011: nextstate = S5_MEMWRITE;  // sw
          default:    nextstate = S0_FETCH;
        endcase
      end

      S3_MEMREAD:  nextstate = S4_MEMWB;
      S4_MEMWB:    nextstate = S0_FETCH;
      S5_MEMWRITE: nextstate = S0_FETCH;
      S6_EXECUTER: nextstate = S7_ALUWB;
      S7_ALUWB:    nextstate = S0_FETCH;
      S8_EXECUTEI: nextstate = S7_ALUWB;
      S9_JAL:      nextstate = S7_ALUWB;
      S10_BEQ:     nextstate = S0_FETCH;
      default:     nextstate = S0_FETCH;
    endcase
  end
```

```
always_comb begin
   alusrca   = 2'b00;
   alusrcb   = 2'b00;
   resultsrc = 2'b00;
   adrsrc    = 1'b0;
   irwrite   = 1'b0;
   regwrite  = 1'b0;
   memwrite  = 1'b0;
   branch    = 1'b0;
   pcupdate  = 1'b0;
   aluop     = 2'b00;

   case (state)
      S0_FETCH: begin
         alusrca   = 2'b00;
         alusrcb   = 2'b10;
         resultsrc = 2'b10;
         adrsrc    = 1'b0;
         irwrite   = 1'b1;
         pcupdate  = 1'b1;
         regwrite  = 1'b0;
         memwrite  = 1'b0;
         branch    = 1'b0;
         aluop     = 2'b00;
      end

      S1_DECODE: begin
         alusrca   = 2'b01;
         alusrcb   = 2'b01;
         resultsrc = 2'b00;
         adrsrc    = 1'b0;
         irwrite   = 1'b0;
         pcupdate  = 1'b0;
         regwrite  = 1'b0;
         memwrite  = 1'b0;
         branch    = 1'b0;
         aluop     = 2'b00;
      end

      S2_MEMADR: begin
         alusrca   = 2'b10;
         alusrcb   = 2'b01;
         resultsrc = 2'b00;
         adrsrc    = 1'b0;
         irwrite   = 1'b0;
         pcupdate  = 1'b0;
         regwrite  = 1'b0;
         memwrite  = 1'b0;
         branch    = 1'b0;
         aluop     = 2'b00;
      end
```

```
S3_MEMREAD: begin
   alusrca   = 2'b00;
   alusrcb   = 2'b00;
   resultsrc = 2'b00;
   adrsrc    = 1'b1;
   irwrite   = 1'b0;
   pcupdate  = 1'b0;
   regwrite  = 1'b0;
   memwrite  = 1'b0;
   branch    = 1'b0;
   aluop     = 2'b00;
end

S4_MEMWB: begin
   alusrca   = 2'b00;
   alusrcb   = 2'b00;
   resultsrc = 2'b01;
   adrsrc    = 1'b0;
   irwrite   = 1'b0;
   pcupdate  = 1'b0;
   regwrite  = 1'b1;
   memwrite  = 1'b0;
   branch    = 1'b0;
   aluop     = 2'b00;
end

S5_MEMWRITE: begin
   alusrca   = 2'b00;
   alusrcb   = 2'b00;
   resultsrc = 2'b00;
   adrsrc    = 1'b1;
   irwrite   = 1'b0;
   pcupdate  = 1'b0;
   regwrite  = 1'b0;
   memwrite  = 1'b1;
   branch    = 1'b0;
   aluop     = 2'b00;
end

S6_EXECUTER: begin
   alusrca   = 2'b10;
   alusrcb   = 2'b00;
   resultsrc = 2'b00;
   adrsrc    = 1'b0;
   irwrite   = 1'b0;
   pcupdate  = 1'b0;
   regwrite  = 1'b0;
   memwrite  = 1'b0;
   branch    = 1'b0;
   aluop     = 2'b10;
end
```

```
S7_ALUWB: begin
   alusrca   = 2'b00;
   alusrcb   = 2'b00;
   resultsrc = 2'b00;
   adrsrc    = 1'b0;
   irwrite   = 1'b0;
   pcupdate  = 1'b0;
   regwrite  = 1'b1;
   memwrite  = 1'b0;
   branch    = 1'b0;
   aluop     = 2'b00;
end

S8_EXECUTEI: begin
   alusrca   = 2'b10;
   alusrcb   = 2'b01;
   resultsrc = 2'b00;
   adrsrc    = 1'b0;
   irwrite   = 1'b0;
   pcupdate  = 1'b0;
   regwrite  = 1'b0;
   memwrite  = 1'b0;
   branch    = 1'b0;
   aluop     = 2'b10;
end

S9_JAL: begin
   alusrca   = 2'b01;
   alusrcb   = 2'b10;
   resultsrc = 2'b00;
   adrsrc    = 1'b0;
   irwrite   = 1'b0;
   pcupdate  = 1'b1;
   regwrite  = 1'b0;
   memwrite  = 1'b0;
   branch    = 1'b0;
   aluop     = 2'b00;
end

S10_BEQ: begin
   alusrca   = 2'b10;
   alusrcb   = 2'b00;
   resultsrc = 2'b00;
   adrsrc    = 1'b0;
   irwrite   = 1'b0;
   pcupdate  = 1'b0;
   regwrite  = 1'b0;
   memwrite  = 1'b0;
   branch    = 1'b1;
   aluop     = 2'b01;
end
```

```
        default: begin
          alusrca   = 2'b00;
          alusrcb   = 2'b00;
          resultsrc = 2'b00;
          adrsrc    = 1'b0;
          irwrite   = 1'b0;
          regwrite  = 1'b0;
          memwrite  = 1'b0;
          branch    = 1'b0;
          pcupdate  = 1'b0;
          aluop     = 2'b00;
        end
      endcase
    end

endmodule
```

ALUDEC

```
module aludec(
    input  logic      opb5,
    input  logic [2:0] funct3,
    input  logic      funct7b5,
    input  logic [1:0] aluop,
    output logic [2:0] alucontrol
);

    logic rtypesub;

    assign rtypesub = funct7b5 & opb5;

    always_comb begin
      case (aluop)
        2'b00: alucontrol = 3'b010; // add
        2'b01: alucontrol = 3'b110; // sub
        default: begin
          case (funct3)
            3'b000: alucontrol = rtypesub ? 3'b110 : 3'b010; // sub / add-addi
            3'b010: alucontrol = 3'b111; // slt
            3'b110: alucontrol = 3'b001; // or
            3'b111: alucontrol = 3'b000; // and
            default: alucontrol = 3'b010;
          endcase
        end
      endcase
    end

endmodule
```

INSTRDEC

```
module instrdec(
   input  logic [6:0] op,
   output logic [1:0] immsrc
);

   always_comb begin
      case (op)
         7'b0110011: immsrc = 2'bxx; // R-type
         7'b0010011: immsrc = 2'b00; // I-type ALU
         7'b0000011: immsrc = 2'b00; // lw
         7'b0100011: immsrc = 2'b01; // sw
         7'b1100011: immsrc = 2'b10; // beq
         7'b1101111: immsrc = 2'b11; // jal
         default:    immsrc = 2'b00;
      endcase
   end

endmodule
```

Q3)

Debug & Design Note: Resolution of ALU Specification Mismatch

In the initial stage of my design, I implemented the aludec.sv module strictly following Table 1 from the lab manual. However, during simulation, I encountered multiple errors in the transcript. To debug this, I analyzed the controller.tv file and the simulation waveforms.

I identified that the errors were caused by a mismatch between the lab manual and the testbench:

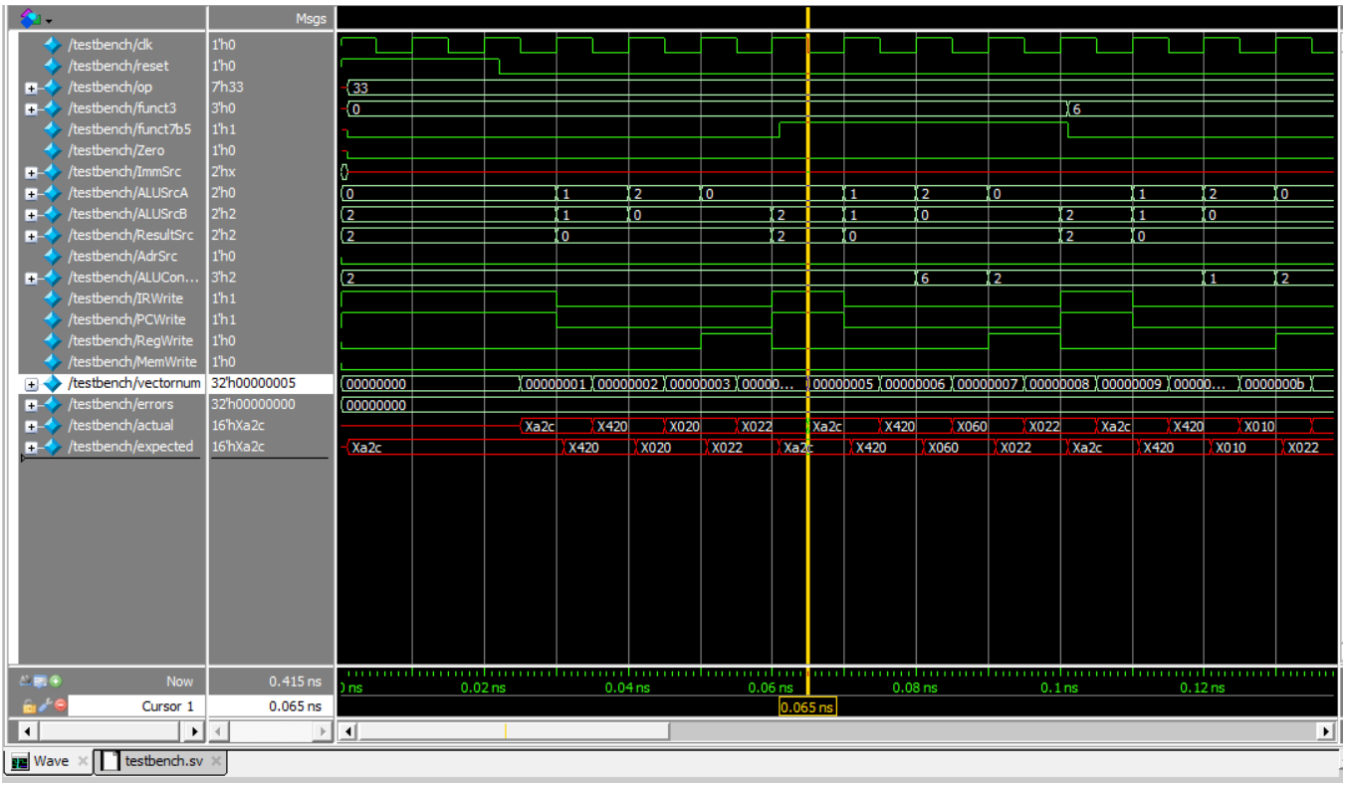
Table 1 uses 000 for ADD and 001 for SUB.

The testbench (controller.tv) expects 010 for ADD and 110 for SUB.

By identifying this conflict, I realized that the testbench follows the universal RISC-V standards. I then updated my aludec.sv module to match these standard codes. This debug process resolved all mismatches, and the final simulation successfully passed with 0 errors.

Yes, the implemented controller successfully passes all test vectors provided in the controller.tv file without any errors.

```
#
# add wave *
# view structure
# .main_pane.structure.interior.cs.body.struct
# view signals
# .main_pane.objects.interior.cs.body.tree
# run -all
#         40 tests completed with          0 errors
# ** Note: $stop    : C:/fpga_projelerim/controller/testbench.sv(81)
#    Time: 415 ps  Iteration: 1  Instance: /testbench
# Break in Module testbench at C:/fpga_projelerim/controller/testbench.sv line 81
```



The testbench samples the outputs at the negative clock edge (0.065 ns). As seen in the waveform, the controller's actual output perfectly matches the expected value (16'hXa2c). This confirms that the FSM state transitions and decoder logics are timed and functioning exactly as required.

Q4)

GitHub Link: https://github.com/berkeee112/RISC-V-Multicycle-Controller.git